

CHAPTER THREE

Design and Implementation

3.1 Design and Implementation

The methodology adopted for this project revolves around modern web development practices, specifically utilizing a component-based architecture via React.js and a robust backend using the MERN stack (MongoDB, Express.js, React, Node.js). The primary design philosophy of the website builder is to offer a "Hybrid Capabilities" model. Traditional builders generally fall into two categories: strict grid/flow-based (like Webflow) and absolutely free-drag (like Wix). In this system, both approaches are synthesized. Why was this done? To provide advanced users with the structured capabilities of CSS Flexbox and Grid for responsive sections while still retaining the freedom to drag and drop overlapping elements absolutely using X and Y coordinates.

3.2 General Structure

The system's architecture follows a classic Client-Server (Frontend-Backend) model:

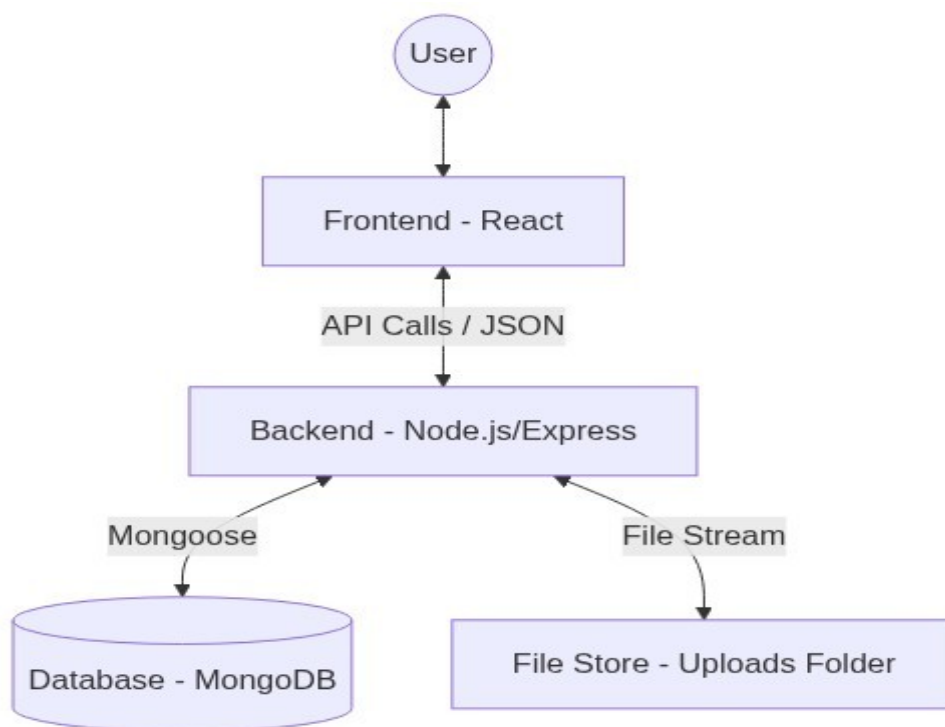


Figure 1: illustrated in the following high-level system architecture diagram.

- **Frontend (Client):** Built exclusively as a Single Page Application (SPA). The state of the entire project (including multiple pages, nested HTML components, and deeply modified CSS styles) is held in memory using React Hooks (`useState`, `useCallback`, `useEffect`). Global context providers (`AuthContext`, `DndProvider`) wrap the application to manage session states and drag-and-drop contexts efficiently.

```
// App.js - Entry Point & Providers
```

```
return (  
  <AuthProvider>  
    <DndProvider backend={HTML5Backend}>  
      <Editor />  
    </DndProvider>  
  </AuthProvider>  
);
```

- **Backend (Server):** A Node.js environment running Express handles RESTful API requests. The choice to use a NoSQL database (MongoDB) was critical here. Since a user's website layout is essentially a massive, varying JSON tree containing elements and their styles, saving this data in rigid SQL tables would require complex, slow joins. MongoDB naturally accepts these JSON structures directly into the Project collection (`data` field), making load and save operations remarkably fast.

```
// Project.js Model - NoSQL JSON Storage
```

```
const ProjectSchema = new mongoose.Schema({
  title: String,
  data: {
    type: Object, // Stores the massive JSON layout tree
    default: {}
  }
});
```

3.2.1 Stateless Authentication (JWT)

To secure the Client-Server communication without degrading the performance of the SPA, Stateless Authentication using JSON Web Tokens (JWT) was implemented. Unlike traditional Session-ID approaches that force the server to repeatedly query the database (increasing server overhead), JWTs carry the user's verified claims (like `userId`) inside a cryptographically signed payload [14]. This ensures data integrity and scalability, allowing the frontend to quickly verify user sessions mathematically without waiting on database latency [15].

```
// server/utils/authUtils.js - Stateless Token Generation
```

```
const generateToken = (userId) => {
  // Encapsulate the userId into a signed payload
  return jwt.sign({ id: userId }, process.env.JWT_SECRET, {
    expiresIn: '30d' // Token remains valid for 30 days
  });
};
```

Communication between the two layers occurs asynchronously via Axios, with an automated debounced saving mechanism introduced on the frontend to prevent data loss without overwhelming the server with continuous requests on every pixel movement.

3.3 User Interfaces and Pages

The system is heavily modularized, rendering different "pages" or "views" based on user interaction securely bounded by authentication.

3.3.1 User Page (Authentication and Dashboard)

Access to the builder tool requires user identification. The `AuthPage` serves as the gateway, handling user registration and login securely.

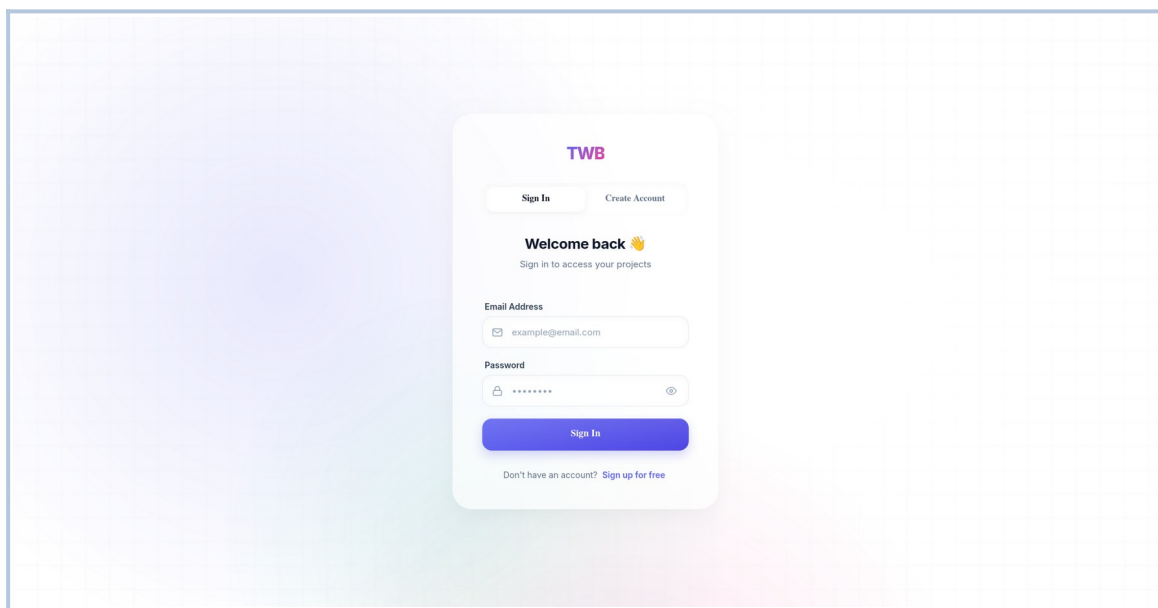


Figure 2: User Authentication.

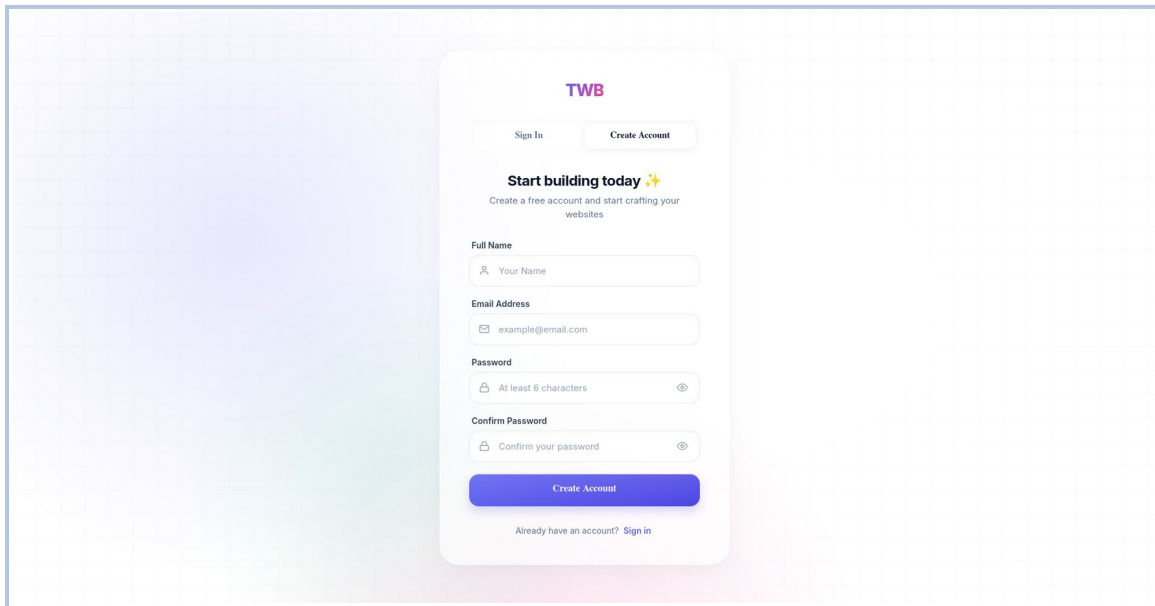


Figure 3: User Registration Interface.

Design Justification: We implemented an authentication barrier (via `PrivateRoute` bindings) because website data is persistent and personal. By tying a `userId` to a `Project` document in MongoDB, the system ensures data privacy and allows users to seamlessly resume their work from any device. A dropdown User Menu was also designed to give quick access to account details and sign-out functionality.

Implementation Logic (Frontend Validation): Before the data is transmitted to the server, the frontend performs a deterministic check to ensure data integrity:

// client/src/pages/AuthPage/AuthPage.js - Validation Logic

```
if (formData.password !== formData.confirmPassword) {  
  setLocalError('Passwords do not match');  
  setIsLoading(false);  
  return; }  

```

Implementation Logic (Backend Authentication): The following server-side controllers manage the security lifecycle of a user session, from initial registration to secure login verification.

// server/controllers/authController.js - Registration & Login Logic

```
const register = async (req, res) => {
  const { name, email, password } = req.body;
  const existingUser = await User.findOne({ email });
  if (existingUser) return res.status(400).json({ message: 'Email already
registered' });

  const user = await User.create({ name, email, password });
  const token = generateToken(user._id);
  res.status(201).json({ token, user: { id: user._id, name, email } });
};

const login = async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (user && (await user.matchPassword(password))) {
    const token = generateToken(user._id);
    res.status(200).json({ token, user: { id: user._id, name: user.name,
email } });
  } else {
    res.status(401).json({ message: 'Invalid credentials' });
  }
};
```

To protect private routes, a centralized middleware validates the JSON Web Token (JWT) in the request header:

```
// server/middleware/authMiddleware.js
```

```
const protect = async (req, res, next) => {  
  let token = req.headers.authorization?.split(' ')[1];  
  if (!token) return res.status(401).json({ message: 'No Token' });  
  
  try {  
    const decoded = jwt.verify(token, process.env.JWT_SECRET);  
    req.user = await User.findById(decoded.id).select('-password');  
    next();  
  } catch (err) {  
    res.status(401).json({ message: 'Invalid Session' });  
  }  
};
```

3.3.2 Editor Page

The core workspace (`Editor.js`) is the most computationally heavy page in the system. It is divided into three functional zones:

1. **The Sidebar/Toolbar:** Contains draggable elements categorized logically (Basic, Media, Sections) and a Page Manager. The Page Manager allows users to add, rename, or delete multiple sub-pages (e.g., Home, Contact) within a single project, dynamically updating the React state without refreshing the browser.
2. **The interactive Canvas:** Acts as the drop zone (`useDrop`). It calculates exact mouse coordinates, applying an optional "Grid Snapping" math algorithm to align elements perfectly.
3. **The Properties Panel:** A dynamic panel that reflects the CSS properties of the currently selected element.

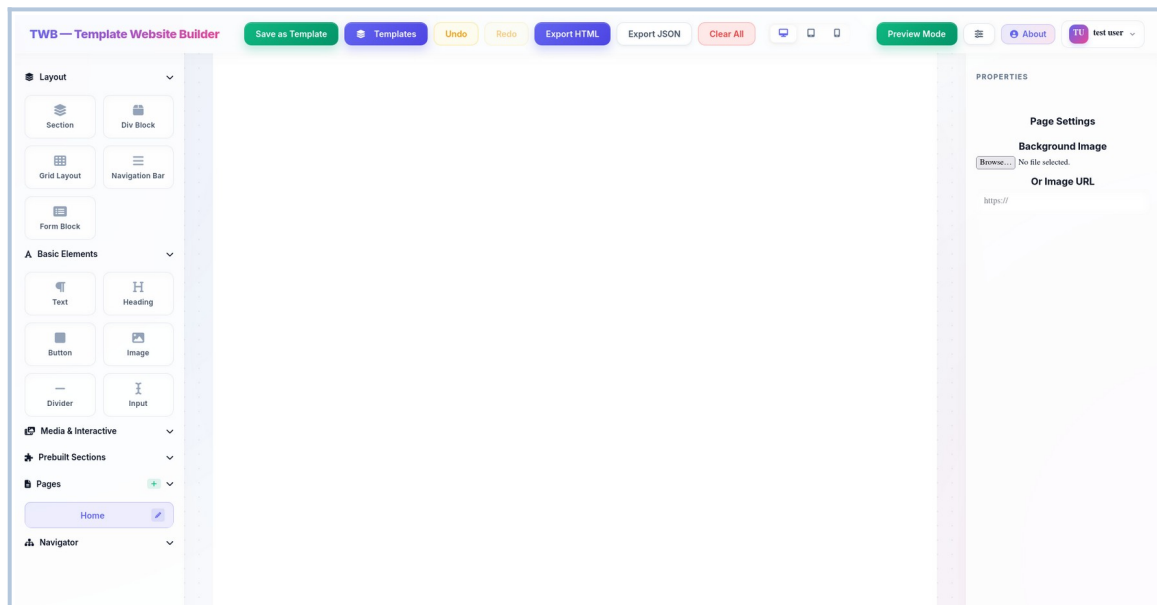


Figure 4: Full Editor Workspace Structure (Sidebar, Canvas, Properties).

Design Justification: To improve User Experience (UX), a floating toolbar was implemented over selected elements, allowing quick actions like duplication or deletion without having to travel across the screen to the sidebar. Furthermore, an Undo/Redo stack (history and future states) was explicitly built to track changes to the canvas state, forgiving user mistakes instantly.

Implementation Logic (Editor Engine): The Editor's performance and alignment accuracy are maintained via several low-level algorithmic implementations:

1. **Grid Snapping and Drop Calculations:** To ensure geometric precision, the useDrop handler intercepts raw pointer coordinates and rounds them to the nearest grid step.

// client/src/components/Editor/Editor.js - Grid Snapping Logic

```
const gridSize = 10;
let x = (clientOffset.x - canvasRect.left - borderLeft) + scrollLeft;
let y = (clientOffset.y - canvasRect.top - borderTop) + scrollTop;
x = Math.round(x / gridSize) * gridSize;
y = Math.round(y / gridSize) * gridSize;
```

2. Interactive State Management (Hover/Active): Components dynamically resolve their styles based on user interactions, allowing for a "Live" design experience.

// client/src/components/Editor/CanvasItem.js - Style Resolution Engine

```
const getActiveStyle = () => {
  let finalStyle = { ...comp.style };
  if (isHovered && comp.states?.hover) finalStyle =
  { ...finalStyle, ...comp.states.hover };
  if (isActive && comp.states?.active) finalStyle = { ...finalStyle, ...comp.states.active };
  return finalStyle;
};
```

3. Multi-Device Responsive Logic: Styles are isolated per breakpoint, ensuring that mobile-specific changes do not affect the desktop layout.

// client/src/components/Editor/Editor.js - Responsive Breakpoint Logic

```
const currentStyle = viewMode === 'desktop'
  ? comp.style
  : { ...comp.style, ...(comp.responsiveStyles?.[viewMode] || {}) };
```

4. Debounced Persistence (Auto-Save): To prevent server overload while ensuring data safety, an automated saving mechanism triggers 1000ms after the last canvas modification.

// client/src/components/Editor/Editor.js - Debounced Persistence Logic

```
useEffect(() => {  
  
  const debounceTimer = setTimeout(() => {  
  
    saveProjectToServer(pages);  
  
  }, 1000);  
  
  return () => clearTimeout(debounceTimer);  
  
}, [pages]);
```

3.3.3 Preview Page (Mode)

Rather than redirecting the user to a completely separate URL to view their site, a previewMode state toggle is built directly into the Editor.

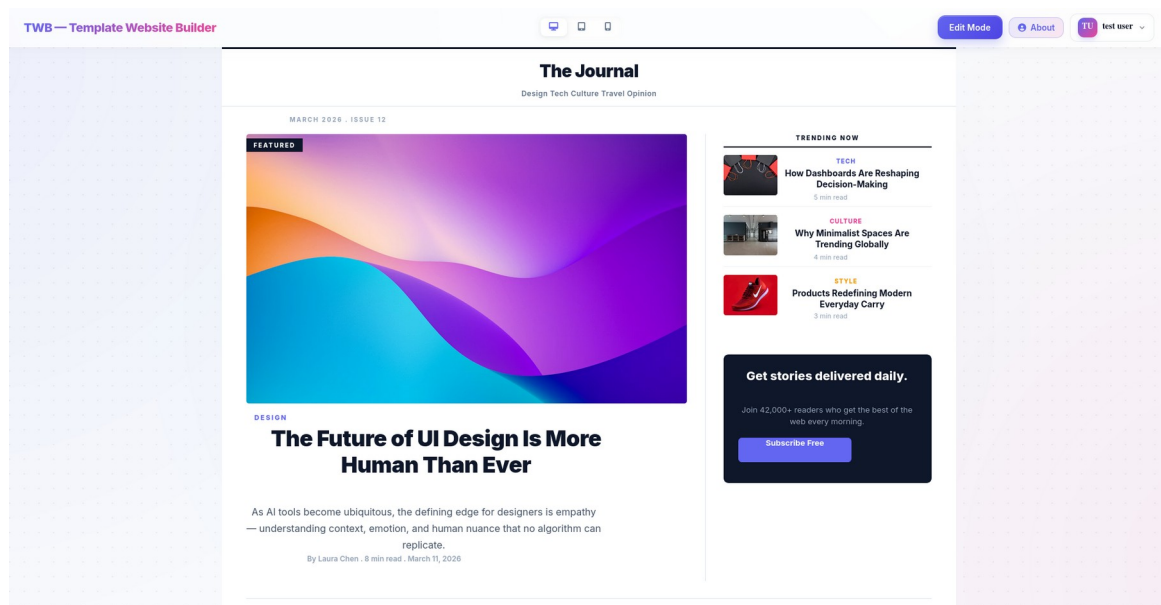


Figure 5: Preview Mode simulating final website rendering.

Design Justification: Activating Preview Mode completely hides the sidebar, properties panel, bounding boxes, and grid lines. This decision was made to drastically reduce the feedback loop time. The user can see the exact, final rendered HTML/CSS output instantly, assessing responsive design modes (Desktop, Tablet, Mobile) without the heavy loading times of a traditional external preview link.

Implementation Logic (Export Engine): The platform utilizes a serialization logic that compiles the complex React state into semantic, standalone HTML/CSS.

1. Dynamic HTML Generation: This recursive function traverses the component tree, converting style objects into inline CSS and assigning unique IDs for targeted styling.

// client/src/utils/exportUtils.js - Recursive HTML Serializer

```
const generatePreviewHTML = (components) => {  
  return components.map(comp => {  
    const styleStr = Object.entries(comp.style).map(([k, v]) =>  
      `${k.replace(/[A-Z]/g, m => '-' + m.toLowerCase())}:${v}`  
    ).join(';');  
    return `  }).join('\n');  
};
```

2. Production Export: The system bundles the generated HTML with a consolidated CSS block and triggers a browser download using a Blob object, making the design portable.

// client/src/components/Editor/Editor.js - Production Export Engine

```
const handleExportHTML = () => {
```

```

const fullHTML = `<!DOCTYPE html>...<style>${styles}</style><body>${body}</body></html>`;

const blob = new Blob([fullHTML], { type: 'text/html' });

const url = URL.createObjectURL(blob);

const link = document.createElement('a');

link.href = url;

link.download = 'exported_site.html';

link.click();
};

```

3.3.4 Templates Page (Template Gallery)

The Template Gallery is integrated as a modal/overlay page that presents users with pre-designed layouts.

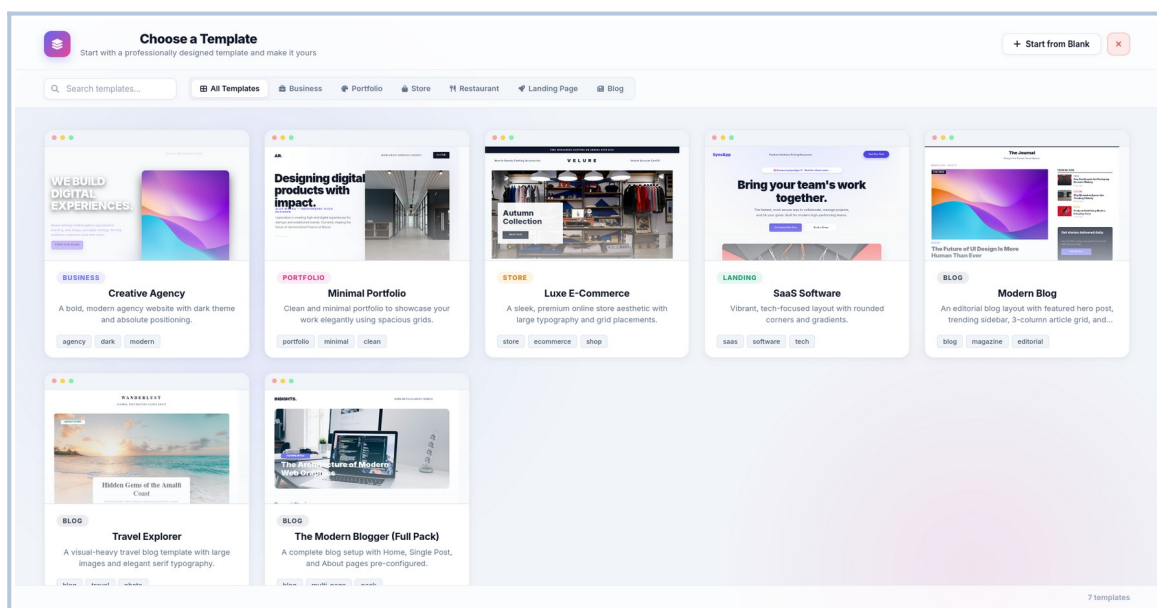


Figure 6: The internal Template Gallery selection screen.

Design Justification: Analyzing user retention in drag-and-drop builders revealed that users often face "blank canvas paralysis." By providing a database-driven `TemplateCollection`, users can inject professionally designed, pre-built component arrays directly into their workspace. This jumpstarts the design process and acts as a learning tool for how complex grids and flexboxes can be constructed.

Implementation Logic (Templates & Portability): Saving and loading templates involves a transformation layer that ensures newly injected components receive unique IDs and consistent positioning.

1. Template Persistence: Layouts are stored as raw JSON objects within a shared MongoDB collection.

// client/src/components/Editor/Editor.js - Template Database Persistence

```
const saveTemplate = async (templateData) => {  
  await axios.post('/api/templates', {  
    name: templateData.name,  
    components: templateData.activeComponents  
  });  
};
```

2. Component Normalization: When a template is loaded, IDs are regenerated based on timestamps to prevent collisions within the current project state.

// client/src/utils/templateUtils.js - Component Normalization

```
const normalizeComponents = (comps) => {  
  return comps.map((comp, idx) => ({  
    ...comp,  
    id: Date.now() + idx, // Ensure global uniqueness  
    style: { ...comp.style, position: 'absolute' }  
  }  
  )  
};
```

```
    }));  
};
```

3.3.5 About Page

The AboutPage serves as informational and instructional documentation accessible directly from the application workspace.

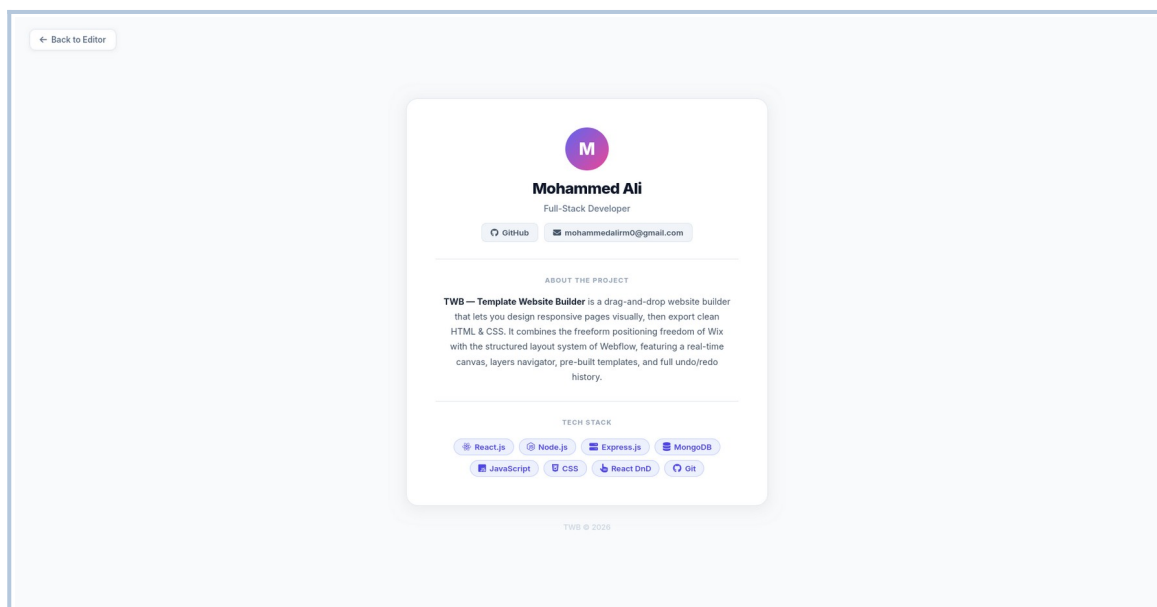


Figure 7: About Page.

Design Justification: Instead of hosting documentation externally, placing the About page within the SPA routing ensures that users can quickly learn about system, or project details without disrupting their session flow or losing their unsaved progress.

3.4 State Management and Algorithmic Logic

3.4.1 React State Management

Handling the state of an entire website layout within a browser's memory is computationally expensive. The state of the project (multiple pages, nested HTML components, and deeply modified CSS properties) is held using React Hooks (`useState`, `useCallback`) and Global Contexts. Instead of using heavy global stores like Redux, which increase bundle size and memory footprint, the local component tree manages the updates.

// client/src/components/Editor/Editor.js - Complex State Structure & Update Pattern

```
const [pages, setPages] = useState([  
  id: 'home',  
  name: 'Home',  
  components: [], // Array of CanvasItem objects  
  style: { backgroundColor: '#ffffff' }  
]);  
  
const updateComponentStyles = useCallback((id, newStyles) => {  
  setPages(prevPages => prevPages.map(page => ({  
    ...page,  
    components: page.components.map(comp =>  
      comp.id === id ? { ...comp, style: { ...comp.style, ...newStyles } } :  
comp
```



```
    )  
    }));  
  }, [activePageId]);
```

The use of the Virtual DOM ensures that when a user drags an element or types in a text box, React only re-renders the specific component (`CanvasItem`) that changed, preserving the 60-FPS smoothness required for a dragging interface [16].

3.4.2 Grid Snapping and Drop Calculations

The drag-and-drop mechanism is not purely visual; it involves an underlying spatial algorithm. When a user drags an element, the `useDrop` hook calculates the exact pointer coordinates (`X, Y`) relative to the `Canvas` bounding box. To ensure structural alignment, a Grid Snapping algorithm intercepts these coordinates before rendering:

```
x = Math.round(x / gridSize) * gridSize
```

```
// client/src/components/Editor/Editor.js - Spatial Coordinate Calculation Logic
```

```
const [, drop] = useDrop({  
  accept: 'ITEM',  
  hover(item, monitor) {  
    const clientOffset = monitor.getClientOffset();  
    const canvasRect = canvasRef.current.getBoundingClientRect();  
  
    // Calculate raw position  
    let rawX = clientOffset.x - canvasRect.left;
```

```
let rawY = clientOffset.y - canvasRect.top;

// Apply Mathematical Snapping Logic
const snappedX = Math.round(rawX / gridSize) * gridSize;
const snappedY = Math.round(rawY / gridSize) * gridSize;

moveComponent(item.id, snappedX, snappedY);
}
});
```

This mathematical coercion forces randomly dragged elements into a strict, invisible geometric grid (e.g., 10px by 10px), eliminating the misalignment issues commonly found in absolute free-drag editors.